

Code-Layout, Readability and Reusability

Date	29 June 2026
Team ID	
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the projectCode Layout & Readability Structure: is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform 4-space tabs in Python (PEP 8), 2-space in JSX/React	Yes	PEP 8 and ESLint compliant
2	Proper File Structure	Files organized into backend/app/routers/, frontend/src/pages/, components/, context/, services/	Yes	Clean separation of concerns
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Self-documenting code
4	Function / Method Names	Functions are descriptively named	Yes	Follows Python and JS naming conventions
5	Code Comments	Inline and block comments explain log	Yes	Comments present in financial_engine.py, ai_service.py, and all router files
6	Modular Design	Separate modules: routers/ (API endpoints), financial_engine.py (calculations), ai_service.py (Gemini), models.py (ORM), schemas.py (validation)	Yes	High modularity across backend and frontend
7	No Redundant Code	Duplicate or unused code is removed	Yes	Clean codebase

8	Error Handling	Try-except blocks for Gemini API calls, database operations, JWT validation, and input parsing	Yes	Graceful fallback to templates on AI failure
---	----------------	--	-----	--

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technolog	Where Reused	Reusability Level (High / Medium / Low)
1	Financial Engine (financial_engine.py)	Python	Financial router, Settlement router, Dashboard router	High

2	AI Service with Fallback (ai_service.py)	Python / Google Gemini	AI Negotiation router, future chatbot features	High
3	Auth Module (auth.py)	Python / JWT + bcrypt	All protected API endpoints via dependency injection	High
4	Axios API Service (api.js)	JavaScript / Axios	All React pages (Dashboard, Loans, Settlements, AI Negotiation, etc.)	High
5	ProtectedLayout Component	React.js	Dashboard, Loans, Financial Health, AI Negotiation, Settlements pages	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Well-organized full-stack architecture with clear module separation
Readability	4	Meaningful names, consistent formatting, and descriptive comments
Reusability	5	Financial engine, AI service, and auth module are highly reusable across all routes
Documentation / Comments	4	Comprehensive docstrings in Python modules and inline comments in financial formulas
Overall Score	4.25	